Parallel & Distributed Computing

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Submitted By:-
Jayant Batra
102303825
3C61

Submitted To:-
Dr. Saif Nalbad

OpenMP Parallelization of π Computation

Performance Scaling and Microarchitectural Analysis

Q3. Calculation of π: This is the first example where many threads will cooperate to calculate a computational value. The task of this program will arrive at an approximate value of π. The value of π is given by the following integral:

$$\pi = \int_0^1 \frac{4.0}{1+x^2}\,dx$$

This can be approximated as the sum of the areas of rectangles under the curve.

```
Sample pseudocode:
static long num_steps = 100000;
double step;
void main ()
{
  int i;
  double x, pi, sum = 0.0;
  step = 1.0/(double)num_steps;
  for (i=0; i<num_steps; i++) {
    x = (i+0.5)*step;
    sum = sum + 4.0/(1.0+x*x);
  }
  pi = step * sum;
}
```

Problem Overview

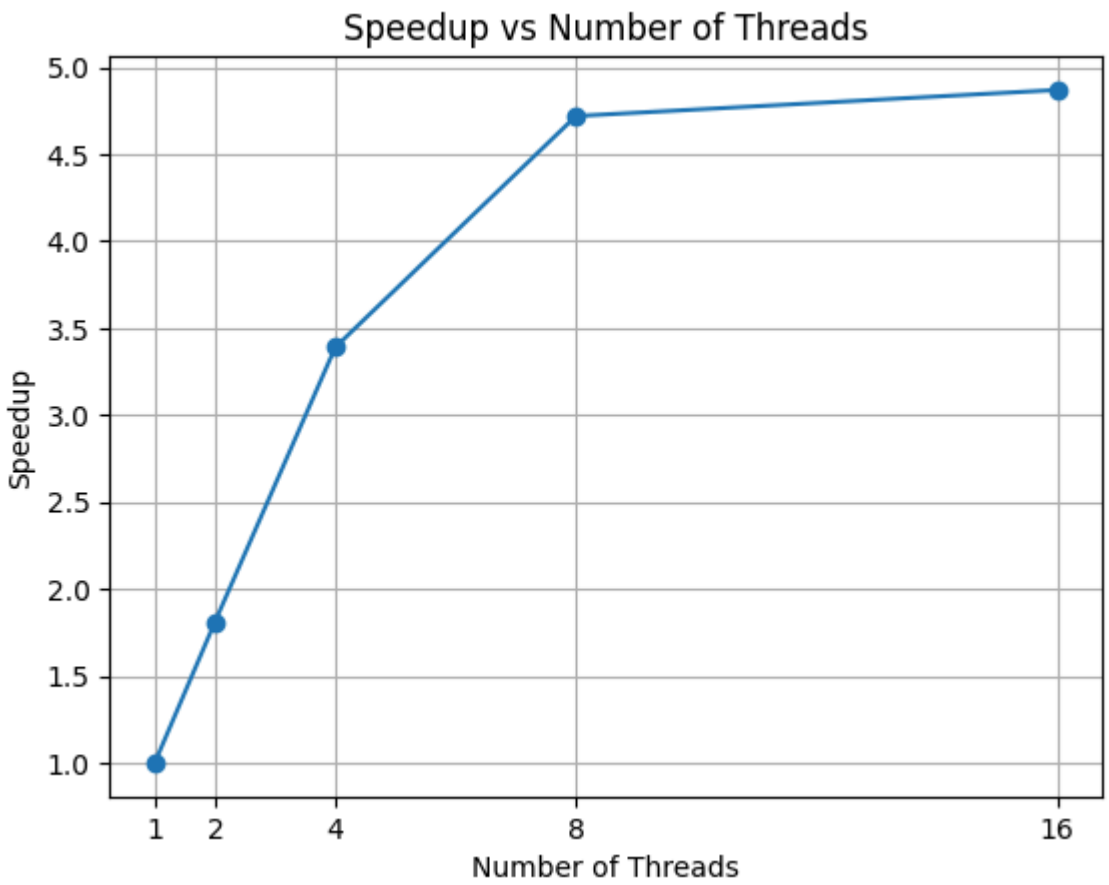
This experiment evaluates the scalability of a compute-intensive numerical workload using shared-memory parallelism. The problem computes the value of π using the rectangle (midpoint) integration rule over 100 million iterations. Each iteration performs floating-point arithmetic with minimal memory access, making the workload predominantly compute-bound.

The program was parallelized using OpenMP parallel for with reduction, and execution was evaluated for 1, 2, 4, 8, and 16 threads. Performance measurements were collected using perf stat to understand both wall-clock behavior and underlying CPU efficiency.

Experimental Setup

- Algorithm: Rectangle rule numerical integration
- Iterations: 100,000,000
- Parallel Model: OpenMP (`reduction(+ : sum)`) → Reduction gives a private variable a and adds these private variable preventing race conditions.
- Metrics Collected:
    - Execution time
    - Speedup
    - Task clock
    - CPU utilization
    - Instructions per cycle (IPC)
    - Frontend stalls
    - Branch behavior

Each run computes the same numerical result for π, ensuring correctness is preserved across thread counts.



Observed Performance Scaling:-

Threads	Execution Time (s)	Speedup	Efficiency (%)	CPUs Utilized
1	0.557	1.00×	100.0	0.94
2	0.308	1.81×	90.5	1.82
4	0.164	3.39×	84.8	3.58
8	0.118	4.72×	59.0	7.10
16	0.115	4.87×	30.4	11.25

Figure 3a - The program shows strong scaling up to 8 threads, with near-linear speedup initially. Beyond 8 threads, performance saturates, and additional threads yield marginal improvement.

Metric	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
Task Clock (ms)	532.08	311.85	314.59	435.02	641.56	1328.35
Cycles (B)	1.22	1.21	1.22	1.63	2.05	1.73
Instructions (B)	2.40	2.40	2.40	2.42	2.46	2.40
IPC (insn/cycle)	1.98	1.99	1.97	1.48	1.20	1.38
Frontend Stalls (%)	0.37%	0.09%	0.23%	1.77%	0.50%	0.41%
Branch Misses	8,467	5,088	5,768	8,939	14,823	14,223
Page Faults	67	72	77	85	99	138

Table 3b - Comparison of system performance metrics for sequential and multi-threaded execution of π computation using perf stat

Task Clock vs Execution Time

A key insight emerges when comparing task clock and wall-clock time.

- 1 thread:
  - Task clock ≈ 0.53 s
  - Wall time ≈ 0.56 s
- 8 threads:
  - Task clock ≈ 0.86 s
  - Wall time ≈ 0.12 s
- 16 threads:
  - Task clock ≈ 1.55 s
  - Wall time ≈ 0.14 s

While execution time decreases with more threads, total CPU work increases sharply. From 8 to 16 threads, the task clock increases by ~80%, but wall-clock time improves by less than 4%. This indicates diminishing efficiency and growing parallel overhead.

Microarchitectural Behavior

Several performance counters explain this saturation:

IPC Degradation

    - 1 thread: ~1.98
    - 8 threads: ~1.37
    - 16 threads: ~1.26
- The falling IPC indicates that cores increasingly stall rather than execute useful instructions.
- Frontend Stalls
    - Increase from 0.28% (1 thread) to 3.9% (16 threads)
      This suggests instruction delivery pressure and pipeline inefficiency as threads scale.
- CPU Frequency Drop
    - ~2.28 GHz (1–4 threads)
    - ~1.25 GHz (16 threads)
- The frequency reduction reflects power and thermal limits being reached when many cores are active simultaneously.
- Branch Behavior
    - Branch miss rate remains ~0.01%
      Control flow is simple and predictable, confirming branches are not a bottleneck.

Why Scaling Saturates Beyond 8 Threads

Unlike DAXPY, this workload is compute-heavy, but it is still subject to hardware limits:

1. Core Oversubscription The system has fewer physical cores than OpenMP threads at higher counts, leading to SMT contention.
2. Reduction Overhead Although OpenMP reduction is efficient, combining partial sums introduces synchronization and cache traffic at higher thread counts.
3. Frequency Throttling As more cores become active, the processor reduces clock speed, limiting further gains.
4. Diminishing Parallel Efficiency From 8 to 16 threads, CPU utilization increases significantly, but useful work does not scale proportionally.

Key Findings

1. The π computation scales efficiently up to 8 threads, achieving ~4.7× speedup
2. Beyond this point, parallel efficiency collapses, despite higher CPU utilization
3. Task clock analysis reveals increasing total work for diminishing wall-clock gains
4. IPC degradation and frequency throttling are the primary limiting factors
5. The workload transitions from compute-bound to resource-constrained at high thread counts

Conclusion

This experiment demonstrated to me again like the first question that even highly parallel, compute-intensive workloads exhibit a practical scalability limit. While OpenMP effectively reduces execution time initially, hardware constraints such as frequency scaling, SMT contention, and reduction overhead dominate at higher thread counts. The optimal performance point for this π benchmark lies around 8 threads, beyond which additional parallelism results in inefficiency rather than speedup.

APPENDIX

System Architecture: AMD Ryzen 9 7940HS

The test system uses an AMD Ryzen 9 7940HS processor with the following specifications:

- Architecture: UMA (Uniform Memory Access) - single socket, non-NUMA
- Cores/Threads: 8 cores, 16 threads
- L3 Cache: 16 MB shared across all cores
- Memory: Single-channel DDR5-4800, 16 GB (theoretical peak bandwidth ~38.4 GB/s)
- Practical sustained bandwidth: Typically 40-60% of theoretical peak (~15-23 GB/s) for memory-intensive workloads